Low-code et digitalisation des processus métier : bénéfices, limites et facteurs de réussite

Msc Vivien AKSANTI Mulumeoderhwa

Low-code et digitalisation des processus métier : bénéfices, limites et facteurs de réussite

Les organisations cherchent à gagner en efficacité, en réactivité et en maîtrise des coûts. Pour y parvenir, beaucoup misent sur la digitalisation de leurs processus métier. Or le développement logiciel traditionnel, avec ses délais et ses ressources techniques importantes, ne suit pas toujours le rythme des besoins. C'est dans ce contexte que les plateformes **low-code** se sont imposées. Mais jusqu'où peuvent-elles vraiment transformer une organisation, et à quelles conditions ?

1. Digitaliser un processus, ce n'est pas seulement numériser

Selon SAP, la digitalisation consiste à utiliser les technologies numériques pour transformer les opérations, les modèles et l'expérience client. Elle ne se résume donc pas à remplacer le papier par un fichier : elle vise une amélioration globale des méthodes de travail, de la circulation de l'information et de la performance.

Un **processus métier**, lui, est défini par J. N. Gillot comme un ensemble ordonné d'activités, manuelles ou automatisées, entreprises dans un objectif déterminé. Pour Dumas et al. (2018), il mobilise des ressources, des acteurs et des informations afin de produire un résultat créateur de valeur.

Digitaliser un processus métier, c'est donc appliquer les technologies numériques à l'ensemble de ses activités pour en améliorer l'efficacité, la qualité et le suivi : moins de tâches manuelles, plus d'automatisation, un partage facilité de l'information et une meilleure traçabilité.

2. Du développement traditionnel au low-code

Le développement traditionnel repose sur des développeurs spécialisés et sur un cycle classique : analyse, développement, tests, déploiement. Il permet des solutions très personnalisées et robustes, mais au prix de délais et de coûts élevés.

Microsoft définit le **low-code** comme une approche qui utilise un codage minimal pour créer des sites web, des applications et des processus, grâce à des modeleurs visuels en glisser-déposer et à des interfaces en pointer-cliquer. Le low-code occupe une position intermédiaire : il combine des composants visuels prêts à l'emploi avec la possibilité d'ajouter du code lorsque le besoin l'exige, là où le **no-code** s'en passe totalement.

Critère	Développement traditionnel	Low-code	No-code
Niveau de programmation	Élevé	Faible	Aucun
Compétences requises	Développeurs spécialisés	Profils techniques et métiers	Utilisateurs métiers
Temps de développement	Long	Réduit	Très rapide
Personnalisation	Très élevée	Élevée	Limitée
Complexité des projets	Forte	Moyenne	Faible
Maintenance	Complexe	Simplifiée	Simplifiée
Coût	Élevé	Moyen	Faible

Quatre principes directeurs

Selon Microsoft, les plateformes low-code reposent sur quatre principes :

- **Agilité et rapidité** : des composants préconfigurés permettent de développer, tester et améliorer plus vite.
- **Collaboration** : les interfaces visuelles créent un langage commun entre équipes métier et équipes techniques.
- **Expérimentation et amélioration continue** : le prototypage rapide permet d'ajuster les solutions grâce aux retours des utilisateurs.
- **Gouvernance et contrôle** : sécurité, conformité et qualité doivent être encadrées par des règles et des bonnes pratiques.

3. Des usages déjà très répandus

Les plateformes low-code servent à automatiser des processus internes, développer des applications métier, centraliser et exploiter des données, concevoir des tableaux de bord ou améliorer la gestion documentaire. Leur usage le plus courant : remplacer des processus fondés sur des échanges d'e-mails, des fichiers Excel et des tâches manuelles par des applications centralisées et automatisées.

Plusieurs cas illustrent cette efficacité :

- **AXA** a développé en trois mois un portail de gestion et de suivi des courriers d'assurance avec la plateforme OutSystems.
- La **SNCF** a développé des applications internes, par exemple pour la gestion des réservations de véhicules ou l'automatisation de tâches de traduction.
- La **ville de Rotterdam** s'appuie sur Mendix pour déployer de nombreuses applications destinées à ses services aux citoyens.

Zoom sur Microsoft Power Platform

Lancée en 2018, la Power Platform regroupe plusieurs outils complémentaires :

- **Power Apps** : applications métier web et mobiles, connectées à SharePoint, Excel ou Dataverse.
- **Power Automate** : flux de travail automatisés (validations, notifications, relances, traitements de données).
- **Power BI** : analyse et visualisation de données sous forme de tableaux de bord interactifs.
- **Power Pages** : sites et portails web sécurisés, internes ou externes.
- **Copilot Studio** : création d'agents conversationnels enrichis par l'intelligence artificielle.

Son principal atout est son intégration native avec Microsoft 365 (Outlook, Teams, Excel, SharePoint, OneDrive), qui facilite la circulation de l'information et valorise les outils déjà en place.

4. Le Citizen Development : les métiers deviennent acteurs

Le *Citizen Development* consiste à permettre aux collaborateurs métier, moyennant formation et accompagnement, de développer eux-mêmes des solutions numériques pour améliorer leur quotidien. Il ne vise pas à remplacer les équipes informatiques, mais à renforcer la collaboration entre métiers et fonctions techniques.

Ses apports sont clairs : moins de dépendance à l'informatique pour les outils simples, davantage d'innovation issue du terrain, et des équipes IT libérées pour les projets stratégiques et complexes.

Mais il comporte aussi des risques : le **Shadow IT** (applications isolées, peu documentées, difficiles à superviser), les enjeux de sécurité et de conformité quand des données sensibles sont manipulées, et les difficultés de maintenance en cas de départ du concepteur initial.

5. Les bénéfices du low-code pour les processus métier

L'analyse de la littérature et de retours d'expérience terrain met en évidence six bénéfices principaux :

1. **Un gain de temps (time-to-market)** : les composants préconfigurés remplacent l'écriture ligne à ligne.
2. **Une réduction des coûts de développement** : moins de ressources techniques mobilisées, moins de recours à l'embauche ou à l'externalisation.
3. **Une autonomie accrue des équipes métier** : les tâches simples sont déléguées, et ceux qui maîtrisent le processus conçoivent l'outil.
4. **Une maintenance facilitée** : moins de code à écrire permet à d'autres membres de l'équipe de prendre le relais.
5. **Une connexion simplifiée aux autres outils** grâce aux connecteurs intégrés, qui favorisent la centralisation des données.
6. **Une accélération de l'innovation**, dans un contexte de pénurie de développeurs et de délais de développement longs.

Concrètement, ces bénéfices se traduisent par des gains observables : suppression de saisies manuelles répétitives, notifications et relances automatiques, meilleure traçabilité des échanges, tableaux de bord de pilotage en temps réel et automatisation de traitements chronophages, comme l'anonymisation de données personnelles dans le cadre de demandes d'exercice de droits RGPD.

6. Les limites à ne pas ignorer

Le low-code n'est pas adapté à tous les contextes. Il doit être pensé comme un **complément** aux solutions informatiques existantes, non comme un remplacement systématique du développement traditionnel.

- **Des fonctionnalités parfois limitées** pour des besoins spécifiques. Par exemple, accéder aux données d'une application métier propriétaire peut exiger le recours à des API dédiées, car les connecteurs natifs ne suffisent pas.
- **Une dépendance à l'écosystème de l'éditeur** : les évolutions de la plateforme, des connecteurs ou des licences peuvent affecter la pérennité des solutions.
- **Des contraintes de performance** lorsque les volumes de données ou les opérations simultanées augmentent.

- **Une personnalisation parfois restreinte**, car les outils reposent sur des composants prédéfinis. Les besoins très spécifiques en interface, en logique métier ou en intégration peuvent nécessiter des développements complémentaires.
- **Des enjeux de sécurité, de conformité et de documentation**, particulièrement sensibles dans les secteurs réglementés.

7. Six facteurs clés de réussite

La technologie ne fait pas tout. Les retours d'expérience convergent vers six facteurs :

7. **Comprendre en profondeur les besoins métier.** Le low-code permet de développer vite, mais une solution rapide qui ne répond pas à un vrai problème reste inutile. Chaque projet doit débuter par un temps d'échange avec les utilisateurs.
8. **Choisir l'outil adapté.** Intégration au système d'information, richesse fonctionnelle, facilité de prise en main, sécurité et conformité : les critères sont multiples. S'appuyer sur un environnement déjà largement adopté réduit les coûts et les frictions.
9. **Impliquer les utilisateurs tout au long du projet.** Ils aident à définir les besoins, à tester et à repérer rapidement les améliorations à apporter, ce qui rejoint les principes du Citizen Development.
10. **Prévoir un plan d'adoption et d'accompagnement.** Une solution performante n'est pas automatiquement utilisée. Démonstrations, documentation, formations et assistance réduisent les résistances au changement.
11. **Mettre en place une gouvernance adaptée.** Règles de développement, documentation, gestion des accès, supervision : le rôle de l'IT n'est pas de freiner l'innovation, mais de l'encadrer pour éviter le Shadow IT. C'est une condition essentielle dans les environnements réglementés.
12. **Adopter une démarche d'amélioration continue.** Les besoins évoluent ; la facilité de modification propre au low-code permet d'ajuster régulièrement les solutions à partir des retours terrain.

Une méthode de projet itérative (recueil et analyse des besoins, conception, développement, tests et réajustements, accompagnement) s'accorde particulièrement bien avec ces facteurs.

8. Perspectives : l'IA change la donne

L'intégration progressive de l'intelligence artificielle, avec Microsoft Copilot et Copilot Studio, ouvre une nouvelle étape. Ces outils assistent désormais les utilisateurs dans la conception d'applications, la création de flux, l'analyse de données ou la recherche d'information, ce qui rend le développement encore plus accessible.

Cette évolution devrait amplifier le Citizen Development. Elle suppose toutefois un cadre de gouvernance solide pour garantir sécurité, conformité et pérennité. L'avenir de la digitalisation des processus semble ainsi s'orienter vers une collaboration renforcée entre métiers, plateformes low-code et IA.

Conclusion

Les outils low-code constituent un levier stratégique de digitalisation des processus métier : ils accélèrent le développement, réduisent les tâches manuelles, améliorent la circulation de l'information et renforcent l'autonomie des métiers. Mais leur réussite repose autant sur des facteurs **organisationnels et humains** (compréhension du besoin, implication des utilisateurs, accompagnement, gouvernance, amélioration continue) que sur la technologie elle-même.

Références

- Dumas, M., La Rosa, M., Mendling, J., & Reijers, H. (2018). *Fundamentals of Business Process Management*. Springer. https://link.springer.com/book/10.1007/978-3-662-56509-4
- Gillot, J. N. (2008). *The complete guide to business process management*. processus-metier.fr. https://www.processus-metier.fr/
- Triaa, W. (2018). *Gestion agile de processus métier : proposition d'une approche tirée par les compétences*.
- Carroll, N. *L'importance du développement citoyen pour la transformation numérique*.
- Boldiereff, M. *Guide pratique pour développer une pratique de low-code / no-code*.
- Microsoft Learning. *Qu'est-ce que le low-code ? Guide des plateformes low-code*, Microsoft Power Apps. https://www.microsoft.com/fr/power-platform/products/power-apps/topics/app-development/low-code-development-guide
- SAP (2023). *What's the Difference? Digitization vs. Digitalization in Digital Transformation*. https://www.sap.com/resources/digitization-vs-digitalization
- Étude de cas : AXA réalise une transformation numérique plus rapide grâce à OutSystems. https://www.outsystems.com/case-studies/insurance-portal
- La ville de Rotterdam favorise le développement à grande échelle, Mendix. https://www.mendix.com/customer-stories/the-city-of-rotterdam-empowers-development-at-scale/
- Claranet. *Modernisation des applications avec la Power Platform de Microsoft 365*.

À propos de l'auteur

Vivien AKSANTI Mulumeoderhwa est titulaire d'un Master MIAGE (parcours Science du Numérique et de Management) de l'Université de Bordeaux. Spécialisé dans la transformation digitale, il conçoit des solutions d'automatisation et de digitalisation des processus métier avec la Microsoft Power Platform et l'écosystème Microsoft 365. Retrouvez ses publications sur https://15vivien.github.io/